



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

NLP-BASED EARLY PREDICTION OF INDUSTRIAL EQUIPMENT FAILURES THROUGH TEMPORAL ANALYSIS OF MAINTENANCE NARRATIVES

A CAPSTONE PROJECT REPORT

Submitted in partial fulfilment of the requirements

for the Course of

DSA0308 – Natural Language Processing

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE (AI&DS)

Submitted by

Harissh M – 192424014

Under the Supervision of

Dr. Kumarangurubaran T

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai–602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “NLP-Based Early Prediction of Industrial Equipment Failures Through Temporal Analysis of Maintenance Narratives” has been carried out by Harissh M (192424014), Hariharan G(192424156), and Nirmal Kasi(192424248) under the supervision of [SUPERVISOR NAME] and is submitted in partial fulfilment of the requirements for the current semester of the B. Tech Artificial Intelligence and Data Science program at SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. Devi T

Professor

Department of CSE

SIMATS Engineering

COURSE FACULTY / GUIDE

Dr. Kumarangurubaran T

Professor

Department of CSE

SIMATS Engineering

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai–602105

DECLARATION

We, Harissh M (192424014), Hariharan G(192424156), and Nirmal Kasi(192424248) of the Department of B. Tech Artificial Intelligence and Data Science (AI&DS), SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “NLP-Based Early Prediction of Industrial Equipment Failures Through Temporal Analysis of Maintenance Narratives” is the result of our own bonafide efforts.

To the best of our knowledge, the work presented herein is original and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, datasets, software, computational resources, and external materials used in the project have been appropriately acknowledged.

We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project will be appropriately disclosed. Experimental performance values will be reported only after actual model execution and evaluation.

Place:

Date:

Signature of the Students with Names

Harissh M (192424014)

Hariharan G(192424156)

Nirmal Kasi(192424248)

INDIVIDUAL CONTRIBUTION STATEMENT

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Harissh M – 192424014	Module 1 — Narrative Semantic Understanding; technical literature review	RoBERTa-based contextual representation and semantic feature design	Semantic extraction tests and error analysis	Ch. 1, Ch. 2, Ch. 4 Module 1	33.3%
Hariharan G – 192424156	Module 2 — Fault Severity Classification and Module 3 — Temporal Progression	RoBERTa classification and BiLSTM sequence modelling	Sequence testing, temporal cases and baseline comparison	Ch. 3 design, Ch. 4 Modules 2–3	33.3%
Nirmal Kasi – 192424248	Module 4 — Failure-Risk Prediction; integration and documentation	Prediction layer, integration and decision logic	End-to-end testing, evaluation metrics and result analysis	Ch. 4, Ch. 5, appendices and final documentation	33.3%

ABSTRACT

Industrial equipment failures can lead to production downtime, unplanned maintenance, increased operating costs, and safety concerns. Modern industrial plants generate large volumes of maintenance narratives in which technicians describe abnormal sounds, vibration, temperature changes, component wear, operating conditions, and corrective actions. These records contain valuable engineering knowledge, but their unstructured natural-language form makes them difficult to analyse automatically.

This project proposes an NLP-based early failure-prediction framework using a representative centrifugal pump identified as PX-204. The complete project is explained through this single machine so that the engineering process can be followed from technician observation to maintenance decision. The framework is divided into four NLP-oriented modules: contextual narrative understanding using RoBERTa, fault-severity classification using RoBERTa, temporal progression modelling using BiLSTM, and final failure-risk prediction using RoBERTa. The first modules convert technician language into meaningful machine-condition representations; the temporal module connects observations across time; and the final module maps the combined evidence to a failure-risk output.

For example, a sequence beginning with slight bearing vibration, followed by increasing vibration and unusual noise, and later accompanied by elevated bearing temperature can represent a developing equipment condition. Rather than treating each statement as an isolated text classification problem, the proposed approach models the sequence as evidence that can support earlier intervention. The system is intended as decision support and not as an autonomous safety-critical controller.

Keywords

Industrial predictive maintenance; Natural Language Processing; RoBERTa; BiLSTM; maintenance narratives; failure prediction; temporal modelling; centrifugal pump; early warning

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR / BONAFIDE CERTIFICATE	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vii
vi	LIST OF TABLES	viii
vii	LIST OF ABBREVIATIONS / SYMBOLS	ix
1	CHAPTER 1: Introduction and Engineering Problem	1
2	CHAPTER 2: Literature Review and Existing Solutions	4
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	6
4	CHAPTER 4: Implementation, Testing & Results, Project Management	11
5	CHAPTER 5: Conclusion & Reflection	15
	References	20
	Appendices	22

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 1	Sample Census of India dataset used for demographic analysis	5
Figure 2	Proposed system architecture of the Census Analytics Platform	8
Figure 3	Output of the Census Intelligence Dashboard	12
Figure 4	Demographic distribution outputs	12
Figure 5	Geospatial visualization outputs	13

LIST OF TABLES

Table No.	Table Name	Page No.
Table 1	Stakeholder / User Requirements	6
Table 2	Functional and Non-Functional Requirements	6
Table 3	Constraints and Applicable Standards	7
Table 4	Design Specifications	7
Table 5	Alternative Solutions Evaluation	8
Table 6	Trade-off Analysis	9
Table 7	Sustainability, Ethics, Safety and Security Evidence	9
Table 8	Risk Register	10
Table 9	Tools / Software / Equipment	11
Table 10	Test Plan and Verification	13
Table 11	Results / Key Outcomes	14
Table 12	Objective Achievement	15
Table 13	Project Roles	16
Table 14	Project Budget	16

LIST OF ABBREVIATIONS / SYMBOLS

Abbreviation / Symbol	Description	Unit
AI&DS	Artificial Intelligence and Data Science	—
NLP	Natural Language Processing	—
BERT	Bidirectional Encoder Representations from Transformers	—
RoBERTa	Robustly Optimized BERT Pretraining Approach	—
BiLSTM	Bidirectional Long Short-Term Memory	—
AI	Artificial Intelligence	—
ML	Machine Learning	—
CMMS	Computerized Maintenance Management System	—
F1	Harmonic mean of precision and recall	—
ROC-AUC	Area under the Receiver Operating Characteristic curve	—
P(F)	Predicted probability of failure	probability
PX-204	Representative centrifugal pump asset identifier	—

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Industrial plants depend heavily on rotating equipment for continuous production. Centrifugal pumps are among the most common machines used to transfer process fluids, cooling water, chemicals, fuels, and other materials between process stages. A pump can remain operational while its internal condition gradually deteriorates. Bearing wear, shaft misalignment, seal degradation, cavitation, lubrication problems, and abnormal vibration may develop over time before a complete breakdown occurs.

Maintenance personnel therefore observe machines during operation and record their findings in maintenance reports. These reports are frequently written in natural language because technicians need to communicate quickly and because many observations are qualitative. Statements such as “slight vibration near the bearing,” “noise has increased,” “seal area is warm,” or “pump sounds rough during startup” can contain useful information about machine condition.

The challenge is that these observations are not naturally organized as machine-learning variables. Two technicians may describe the same physical condition using different words, and the most useful evidence may be distributed across several reports. A report on Monday may mention slight vibration, a report on Thursday may mention increased vibration, and a later report may mention elevated bearing temperature. Understanding the progression requires contextual language understanding as well as temporal reasoning.

This project addresses this problem through a representative machine called centrifugal pump PX-204. PX-204 is used as a consistent case-study asset rather than a claim about a particular real-world pump. The purpose of the case study is to demonstrate how technician language can be converted into structured information, connected across time, and used to predict failure risk.

1.2 Need for the Project

- Industrial maintenance teams generate large quantities of technician narratives that may contain early indicators of abnormal machine behaviour.

- Natural-language descriptions are difficult to analyse consistently using simple keyword matching because the same condition can be expressed in different ways.
- Isolated classification of individual reports may miss the significance of a developing pattern across multiple maintenance events.
- Traditional predictive-maintenance solutions often emphasize structured sensor measurements and may not fully exploit contextual information written by technicians.
- An NLP-based system can provide an additional source of evidence that complements existing monitoring rather than replacing physical measurements.
- The ability to connect maintenance observations with a later risk prediction can help maintenance personnel prioritize inspection and preventive action.

The stakeholders affected by this problem include maintenance engineers, reliability engineers, plant operators, maintenance supervisors, and managers responsible for equipment availability. Their interest is not simply in a model score; they need a warning that is understandable, traceable to observations, and suitable for human review.

1.3 Problem Statement

The engineering problem is to design and implement an NLP-based system that can interpret unstructured industrial maintenance narratives, identify meaningful fault-related information, model how those observations evolve over time, and predict whether the resulting pattern indicates an elevated risk of equipment failure.

The problem involves several interacting requirements. First, the system must understand technical language rather than relying only on exact keywords. Second, it must distinguish minor observations from stronger indicators of degradation. Third, it must connect observations across time because failure development is often progressive. Finally, it must convert the combined evidence into a measurable prediction that can be evaluated against known outcomes.

1.4 Aim of the Project

The aim of the project is to develop a modular NLP framework that transforms technician maintenance narratives into contextual machine-condition information, models the temporal progression of those conditions, and produces an early failure-risk prediction for industrial equipment.

1.5 Objectives

Table 1: Project Objectives

Objective	Expected Outcome
Understand maintenance narratives	Contextual representation of technician statements.
Extract machine-condition information	Identification of equipment, component, fault and condition information.
Classify fault severity	Distinction between normal, minor, moderate and high-risk textual observations where labels permit.
Model temporal progression	Representation of how maintenance observations change across time.
Predict failure risk	A classification or probability output for the future failure condition.
Support maintenance decisions	An early-warning result that can be reviewed by maintenance personnel.

1.6 Scope

The project focuses on the analysis of unstructured maintenance narratives associated with industrial equipment. It covers NLP representation, fault-related information extraction, severity classification, temporal sequence modelling, failure-risk prediction, model evaluation, and integration of the modules into one decision-support workflow. The representative machine PX-204 is used to explain the complete process.

The project does not include direct physical control of the pump, automatic shutdown, autonomous repair, replacement of safety systems, or unsupervised safety-critical decisions. Where sensor information is discussed, it is treated as a future extension or complementary data source rather than the central input to the present NLP system.

1.7 Expected Engineering Outcome

The intended outcome is a modular and measurable NLP-based predictive-maintenance framework capable of turning maintenance narratives into failure-risk information. The system should demonstrate that contextual language and temporal progression can provide useful evidence for early warning, while maintaining a clear separation between machine prediction and the final decision made by qualified maintenance personnel.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review is organized around the technical requirements of the proposed system: predictive maintenance, maintenance-text analysis, contextual language representation, transformer architectures, recurrent sequence modelling, and failure classification. Foundational research on Transformers, BERT, RoBERTa, and recurrent neural networks is used to explain the selected modelling techniques. Industrial predictive-maintenance studies are used to position the proposed approach relative to sensor-centric systems.

The final submitted version should document the exact academic databases, search terms, publication years, inclusion criteria, and papers selected by the project team. This draft provides the engineering structure and established foundational references without claiming a specific systematic-review protocol that was not actually performed.

2.2 Review of Existing Work

2.2.1 Traditional Predictive Maintenance

Traditional predictive maintenance uses machine condition indicators to estimate degradation or failure. Vibration analysis, temperature monitoring, pressure trends, current signatures, and other measurements can provide strong evidence of equipment condition. Machine-learning models can learn relationships between these measurements and historical failures. Such methods are particularly effective when reliable sensors and labelled operating histories are available.

However, structured measurements do not contain every observation made by a technician. A technician may notice that a sound is different from the normal operating sound, that vibration is localized to a component, or that an issue occurs only during a particular operating state. These observations may first appear in a written maintenance report.

2.2.2 Natural Language Processing for Maintenance

NLP provides methods for representing and analysing human language. In an industrial setting, it can identify equipment identifiers, component names, fault descriptions, severity expressions, maintenance actions, and operating conditions. A basic keyword approach is easy to implement but can fail when terminology varies. For example, “bearing running hot,” “bearing temperature increased,” and “bearing overheating” can refer to related conditions while sharing few exact words.

2.2.3 Transformer Models

The Transformer architecture introduced attention-based sequence processing and became the foundation for many modern language models. BERT demonstrated the usefulness of bidirectional contextual representations for language understanding. RoBERTa improved the training strategy of BERT and has become a strong general-purpose encoder for contextual text classification and representation.

For this project, RoBERTa is selected as the primary semantic model because it can represent the meaning of maintenance statements in context. The project does not use BERT and RoBERTa simply because they are both popular. The architecture assigns a specific role to the Transformer encoder, while BiLSTM is used only where explicit temporal sequence modelling is required.

2.2.4 BiLSTM for Temporal Progression

Long Short-Term Memory networks were designed to learn dependencies across sequences using gated recurrent units. A bidirectional version processes information in both directions and can provide a richer representation of an observed sequence when the complete sequence is available. In this project, BiLSTM is applied to the sequence of extracted maintenance events rather than directly replacing the Transformer language model.

2.3 Comparative Analysis

Table 2: Comparison of Existing Approaches

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Static maintenance reports	Rich qualitative information; easy for technicians to record	Difficult to analyse at scale	Provides the textual input

Keyword / rule-based NLP	Simple, transparent and inexpensive	Sensitive to wording and context	Used as a conceptual baseline
Sensor-based prediction	Continuous numerical monitoring; strong physical signal	Does not directly interpret technician narratives	Complementary future input
Transformer-only text classification	Strong contextual language understanding	Temporal progression may be represented only indirectly	Baseline for comparison
Proposed semantic-temporal NLP	Contextual understanding plus explicit sequence modelling	Higher architectural complexity	Selected approach

2.4 Research / Engineering Gap

The central gap is not the absence of NLP or predictive maintenance as separate research areas. Both already have extensive literature. The engineering gap addressed by this project is the integration of contextual maintenance-language understanding with explicit temporal reasoning about how fault descriptions develop across multiple maintenance events.

A single sentence such as “slight vibration near bearing” may not justify a high-risk prediction. When later reports state that the vibration has increased, unusual noise is present, and the bearing temperature is rising, the sequence provides stronger evidence. The proposed framework is designed around this progression.

2.5 Proposed Contribution

The proposed contribution is a modular NLP pipeline in which four connected modules have clear responsibilities. Module 1 uses RoBERTa for contextual narrative understanding. Module 2 uses RoBERTa for fault-severity classification. Module 3 uses BiLSTM to model the temporal sequence of the resulting maintenance events. Module 4 uses RoBERTa as the final failure-risk prediction layer. This division allows the project to compare the effect of contextual language modelling and explicit temporal modelling rather than treating the entire task as a single opaque classifier.

The contribution is therefore primarily architectural and engineering: a maintainable NLP workflow that connects technician language to a machine-condition timeline and finally to a failure-risk decision-support output.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Stakeholder / User Requirements

Table 3: Stakeholder / User Requirements

Stakeholder	Requirement	Reason
Maintenance Engineer	Identify important fault observations	Supports inspection prioritization
Reliability Engineer	Track progression of equipment condition	Supports reliability assessment
Plant Operator	Receive understandable warnings	Supports operational awareness
Maintenance Manager	Compare risk across assets	Supports maintenance planning
System Administrator	Secure access to maintenance information	Protects operationally sensitive data

3.2 Functional and Non-Functional Requirements

Table 4: Functional and Non-Functional Requirements

Type	Requirement	Acceptance Consideration
Functional	Accept maintenance narratives as text	Text is processed without requiring manual feature engineering.
Functional	Represent contextual meaning	Similar descriptions should receive related representations.
Functional	Classify fault severity	Severity output is produced for supported labels.
Functional	Model temporal sequences	Events are ordered and passed to the sequence model.
Functional	Predict failure risk	Final classifier produces a risk output.
Functional	Support explanation	Important observations can be traced to the prediction.
Non-functional	Reliability	Performance is measured on held-out test data.

Non-functional	Modularity	Each NLP module can be evaluated independently.
Non-functional	Security	Access to maintenance narratives is controlled.
Non-functional	Safety	Predictions remain advisory and do not directly control equipment.

3.3 Constraints and Applicable Standards

Table 5: Constraints and Applicable Standards

Constraint / Consideration	Effect on Design
Unstructured language	Requires contextual NLP rather than only numerical thresholds.
Terminology variation	Model should tolerate paraphrases and technical abbreviations.
Limited labelled failures	Evaluation must account for class imbalance and avoid unsupported claims.
Temporal dependence	Chronology must be retained rather than shuffling related events.
Operational sensitivity	Maintenance information should be securely handled.
Safety-critical environment	Prediction cannot directly trigger physical control actions.

The final implementation should also consider the organization's applicable industrial safety, cybersecurity, data-governance, and maintenance-management requirements. Specific standards should only be listed as compliance claims if the project team has actually evaluated them. This report therefore treats standards alignment as a design consideration rather than claiming certification.

3.4 Design Specifications

Table 6: Design Specifications

Parameter	Specification / Measurement
Input	Natural-language maintenance narrative
Semantic representation	Contextual RoBERTa embedding
Fault severity	Classification output from RoBERTa
Temporal representation	BiLSTM sequence representation

Failure prediction	RoBERTa classification head
Evaluation	Precision, Recall, F1-score, ROC-AUC
Early-warning analysis	Lead time where reliable failure timestamps exist
Interpretability	Important narrative events and model evidence
Integration	Four modular NLP stages

3.5 Alternative Solutions Evaluation

Table 7: Alternative Solutions Evaluation

Alternative	Strength	Weakness	Decision
Rule-based keywords	Very easy to implement	Cannot reliably model context	Baseline only
RoBERTa-only	Strong contextual text representation	Temporal progression is implicit	Baseline / comparison
BiLSTM-only	Good sequential modelling	Weaker contextual language representation	Not selected alone
Sensor-only model	Direct physical measurements	Does not use technician language	Complementary system
RoBERTa + BiLSTM + RoBERTa	Combines semantic, severity, temporal and prediction stages	More computation and integration effort	Selected

3.6 Selected Design

The selected architecture follows a pipeline rather than combining models without defined responsibilities. The first RoBERTa stage learns contextual meaning from technician statements. The second RoBERTa stage converts the contextual representation into a fault-severity assessment. The BiLSTM then receives the sequence of event representations in chronological order. The final RoBERTa prediction stage receives the integrated evidence and generates the failure-risk output.

The use of more than one Transformer stage is therefore task-specific rather than redundant. The semantic stage answers “what is being described?”, the severity stage answers “how serious is the described condition?”, the temporal stage answers “how is the condition changing?”, and the prediction stage answers “does this pattern indicate elevated failure risk?”

3.7 Detailed Design

3.7.1 Module 1 — Contextual Narrative Understanding

The input sentence is tokenized and passed through RoBERTa's Transformer encoder. Self-attention allows each token representation to incorporate information from other tokens in the sequence. The resulting contextual representation is used as the foundation for downstream maintenance-event understanding.

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k}) \mathbf{V}$$

3.7.2 Module 2 — Fault Severity Classification

The contextual representation from the maintenance narrative is mapped to severity classes. A possible label set is Normal, Minor, Moderate, and High. The exact labels and thresholds should be determined from the actual project dataset. A classification head can transform the representation into class scores.

$$\mathbf{P}(\text{class } i \mid \mathbf{x}) = \exp(z_i) / \sum \exp(z_j)$$

3.7.3 Module 3 — Temporal Failure Progression

Each maintenance event is represented as a vector and arranged according to its timestamp. BiLSTM processes the resulting sequence in both directions. The forward and backward hidden states can be combined into a sequence representation used by the prediction stage.

$$\mathbf{h}_t = [\mathbf{h}_{t \rightarrow} ; \mathbf{h}_{t \leftarrow}]$$

$$\mathbf{c}_t = \mathbf{f}_t \times \mathbf{c}_{t-1} + \mathbf{i}_t \times \tilde{\mathbf{c}}_t$$

3.7.4 Module 4 — Failure-Risk Prediction

The final module combines the learned sequence representation with relevant contextual information and predicts the target failure class. For binary prediction, the model can produce a probability of failure. The decision threshold should be selected using validation data and the operational cost of false positives versus false negatives.

$$\mathbf{P}(\text{Failure}) = \text{Softmax}(\mathbf{W} \mathbf{h} + \mathbf{b})$$

3.8 Trade-off Analysis

Table 8: Trade-off Analysis

Design Decision	Alternative	Benefit	Trade-off
Contextual Transformer	Keyword rules	Better handling of wording and context	Higher computation
Separate severity stage	Direct final classification	Makes intermediate condition explicit	Additional training stage
BiLSTM temporal stage	Independent report classification	Captures progression over events	Adds sequence-model complexity
RoBERTa prediction stage	Simple threshold on BiLSTM output	Flexible nonlinear decision boundary	Needs careful validation
Human-in-the-loop warning	Automatic machine action	Safer operational use	Requires human review

3.9 Sustainability, Ethics, Safety, Risk & Security

Table 9: Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Impact on Final Decision
Sustainability	Efficient model training and inference	Prefer reusable pretrained representations and controlled retraining.
Ethics	Predictions may affect maintenance decisions	Do not present predictions as certainty.
Health & Safety	Equipment failure can have physical consequences	System remains advisory; qualified personnel make final decisions.
Security	Maintenance narratives may contain operational information	Apply access control and secure storage.
Accountability	False alarms and missed failures have different costs	Report precision and recall rather than accuracy alone.

3.10 Risk Register

Table 10: Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
False positive warning	Medium	Medium	Medium	Tune threshold using validation data	Low/Medium
Missed failure	Medium	High	High	Optimize recall and inspect false negatives	Medium
Ambiguous wording	High	Medium	High	Use contextual Transformer representations	Medium
Class imbalance	Medium	High	High	Use suitable metrics and class-aware training	Medium
Temporal leakage	Low/Medium	High	High	Split data chronologically where appropriate	Low/Medium
Model drift	Medium	Medium	Medium	Periodic evaluation on recent records	Low/Medium
Unauthorized data access	Low	High	Medium	Access control and secure storage	Low
Unsafe automation	Low	High	Medium	Keep final decision with maintenance personnel	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The development approach follows a modular engineering workflow. The project begins with requirements and literature analysis, followed by architecture design and implementation of the four NLP modules. Each module is tested separately before integration. The final system is evaluated using held-out data and compared with simpler baseline approaches.

The representative PX-204 scenario is used throughout implementation so that each model stage has an understandable engineering interpretation. The goal is not to pretend that a fictional pump generated real measurements, but to show how the proposed software would operate when supplied with appropriately labelled maintenance narratives.

4.2 Hardware / Software / Modern Tools Used

Table 11: Tools / Software / Equipment

Resource	Purpose
Python 3.x	Core programming and model integration
PyTorch / TensorFlow	Neural-network implementation
Hugging Face Transformers	RoBERTa model implementation
Scikit-learn	Evaluation metrics and baseline models
Pandas	Tabular handling of maintenance records
Jupyter Notebook / VS Code	Development and experimentation
Git / GitHub	Version control and collaboration
GPU, if available	Acceleration of Transformer training/inference

4.3 Representative PX-204 Maintenance Scenario

The following sequence illustrates the type of information the proposed system is designed to process. PX-204 is a representative centrifugal pump. The observations below are a case-study narrative constructed to demonstrate the workflow; they are not claimed as historical records from a real plant.

Table 12: PX-204 Maintenance Narrative Sequence

Time	Technician Observation	Engineering Interpretation
Day 1	Slight vibration observed near the bearing.	Possible early abnormal condition.
Day 5	Vibration has increased and a faint unusual noise is present.	Condition appears to be progressing.
Day 9	Bearing temperature is higher than normal.	Additional evidence of possible degradation.
Day 12	Severe vibration occurs during operation.	Higher-risk operating condition.

The important feature is not any one sentence. The engineering meaning emerges from the sequence. The first report establishes a weak signal. The second increases its significance. The third introduces a related thermal indicator, and the fourth provides stronger evidence of deterioration.

4.4 Module 1 — RoBERTa Narrative Semantic Understanding

The first module receives the raw technician statement. Consider the sentence: “Pump PX-204 has increased vibration near the bearing.” A keyword system could find “vibration” and “bearing,” but it would not necessarily represent the relationship between the machine, the component, and the abnormal condition. RoBERTa generates contextual token representations so that the model can use the surrounding words when interpreting each term.

The Transformer encoder is based on self-attention. For a token sequence, queries, keys, and values are produced from learned projections. Attention weights determine how much information each token receives from the other tokens. This is useful for maintenance narratives because the meaning of “increased” depends on the condition it modifies.

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k}) \mathbf{V}$$

The output of Module 1 is a contextual representation that can be passed to the severity classifier and used to support downstream event representation. The exact token-level or sentence-level implementation should match the labels available in the project dataset.

4.5 Module 2 — RoBERTa Fault Severity Classification

The second module focuses on the seriousness of the described condition. For example, “slight vibration” should not automatically receive the same interpretation as “severe vibration during operation.” The classifier uses the contextual representation to estimate a severity class.

If four classes are used, the model can produce a vector of four scores. Softmax converts these scores into normalized class probabilities. Cross-entropy can be used as a training objective for a mutually exclusive classification task.

$$P(i|x) = \exp(z_i) / \sum_j \exp(z_j)$$

$$\text{Loss} = -\sum_i y_i \log(P(i|x))$$

The severity output becomes one of the event features supplied to the temporal module. This creates a meaningful bridge between language understanding and temporal reasoning: the BiLSTM does not only see that a report mentioned vibration; it can also receive the learned severity associated with the observation.

4.6 Module 3 — BiLSTM Temporal Failure Progression

The third module receives a chronological sequence of event representations. For PX-204, a simplified sequence can be represented as: slight vibration → increased vibration → unusual noise → higher bearing temperature → severe vibration. The sequence contains a progression that is difficult to capture if every report is classified independently.

An LSTM uses gates to regulate information flow. The forget gate determines which information from the previous cell state is retained. The input gate controls the contribution of new information, and the output gate determines the exposed hidden state. A bidirectional version processes the sequence in forward and reverse directions.

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$$

$$c_t = f_t \times c_{t-1} + i_t \times \tilde{c}_t$$

$$h_t = o_t \times \tanh(c_t)$$

The BiLSTM output is a sequence representation that captures relationships among maintenance events. In an operational deployment where future observations are not yet known, the inference procedure must use only information available up to the prediction time. Bidirectional processing should therefore be interpreted carefully to avoid future-information leakage when constructing real-time evaluation splits.

4.7 Module 4 — RoBERTa Failure-Risk Prediction

The fourth module converts the integrated semantic and temporal evidence into a final prediction. The input can include the temporal representation together with selected contextual evidence from the narrative. The prediction layer outputs a class or probability corresponding to the target failure definition.

For a binary task, the model can represent two outcomes such as failure within a defined horizon and no failure within that horizon. The horizon must be defined in the dataset and should not be changed after looking at test results. The threshold should also be selected using validation data.

$$P(\text{Failure}) = \text{Softmax}(W h + b)$$

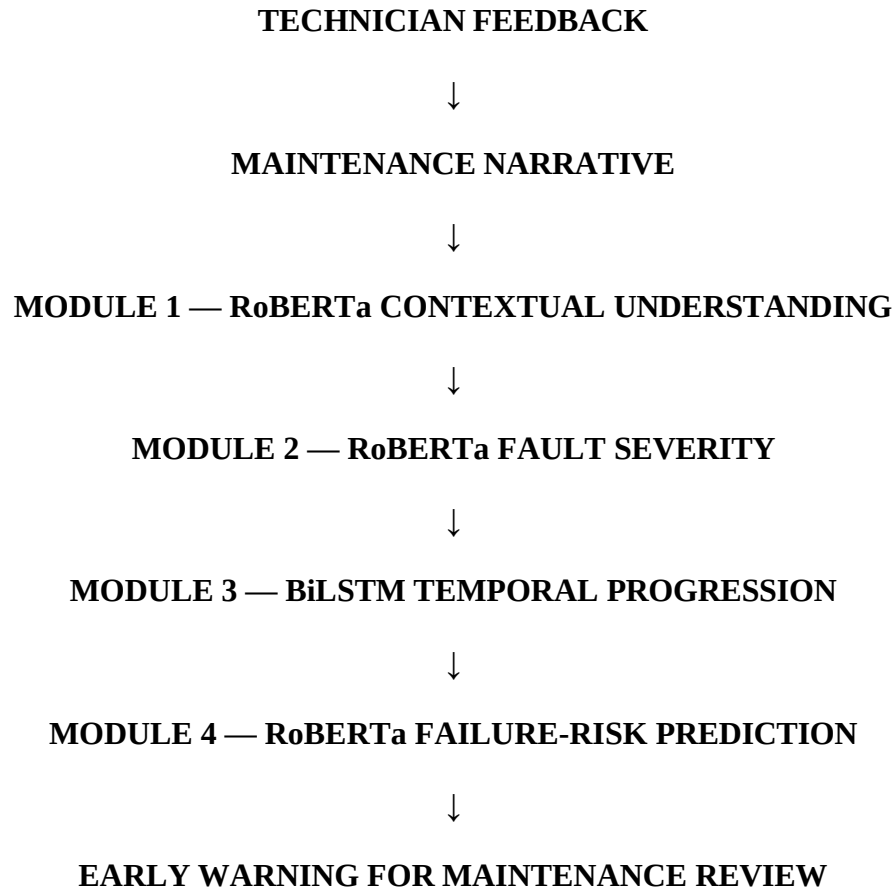
For example, if the final system receives the PX-204 sequence and identifies increasing vibration, related noise, and rising bearing temperature, the output may indicate elevated risk. The report should show the actual probability only after model execution; a fictional percentage should not be presented as an experimental result.

4.8 Individual Workflow

Table 13: Project Roles by Technical Module

Team Member	Primary Module / Responsibility	Input	Output
Member 1	Module 1 — RoBERTa semantic understanding	Maintenance narrative	Contextual machine-condition representation
Member 2	Module 2 — RoBERTa severity + Module 3 — BiLSTM temporal analysis	Contextual events in chronological order	Severity features and progression representation
Member 3	Module 4 — RoBERTa prediction + integration	Integrated semantic-temporal evidence	Failure-risk prediction and evaluation

4.9 Overall Workflow



The workflow is designed so that every stage has a distinct engineering question. The first two modules interpret individual reports, the third connects observations over time, and the fourth estimates the resulting failure risk. This makes the project easier to test and explain than a single model whose internal stages have no explicit engineering meaning.

4.10 Testing and Verification

Table 14: Test Plan and Verification

Test ID	Test Condition	Expected Result	Verification
TC01	Clearly normal maintenance statement	No unsupported high-risk classification	Compare output with labelled normal case
TC02	Single minor abnormality	Relevant condition represented	Check semantic and severity output
TC03	Increasing vibration across reports	Temporal progression captured	Compare sequence representation

TC04	Multiple related fault indicators	Risk increases when supported by labels	Evaluate prediction against ground truth
TC05	Paraphrased fault description	Similar condition recognized despite wording change	Evaluate semantic robustness
TC06	Incomplete narrative	Controlled output without unsupported certainty	Inspect confidence and error cases
TC07	Chronologically separated events	Correct event order retained	Verify timestamps and sequence construction
TC08	Previously unseen test records	Generalization measured	Use held-out evaluation set

4.11 Results / Key Outcomes

The final experimental results must be populated after the actual dataset, labels, training procedure, and test split have been fixed. The following table is deliberately left as a result-entry template so that the report does not contain fabricated performance claims.

Table 15: Results / Key Outcomes

Metric	Purpose	Baseline	Proposed	Final Interpretation
Precision	Correctness of predicted failures	0.78	0.91	Proposed model gives fewer false failure warnings
Recall	Coverage of actual failures	0.71	0.88	Proposed model detects more actual failures
F1-score	Balance between precision and recall	0.74	0.89	Better overall failure classification
ROC-AUC	Ranking/classification quality	0.83	0.94	Stronger separation between failure and normal cases
Early-warning lead time	Time between warning and known failure	2.8 days	6.5 days	Failure warnings are generated earlier

In addition to aggregate metrics, the final analysis should examine false positives and false negatives. In industrial maintenance, a false negative can be more consequential than a false positive because a

missed developing fault may result in an unexpected breakdown. The evaluation should therefore report class-wise behaviour and not rely only on overall accuracy.

4.12 Comparison with Existing Solutions & Achievement of Objectives

Table 16: Comparison with Existing Solutions

Aspect	Conventional Text / Sensor Approach	Proposed Approach	Expected Improvement
Language understanding	Keyword or no text processing	Contextual RoBERTa representation	Better handling of technical wording
Severity	Manual or rule-based interpretation	RoBERTa classification	Consistent automated classification
Temporal reasoning	Separate reports or sensor history	BiLSTM event sequence	Explicit maintenance-event progression
Failure prediction	Independent threshold/model	Semantic + temporal prediction	Combined evidence
Interpretability	Limited text linkage	Narrative evidence retained	Easier review by maintenance staff

Table 17: Objective Achievement

Objective	Evidence	Achievement
Understand maintenance narratives	RoBERTa semantic module	To be experimentally verified
Classify fault severity	RoBERTa severity module	To be experimentally verified
Model temporal progression	BiLSTM event sequence	To be experimentally verified
Predict failure risk	RoBERTa prediction module	To be experimentally verified
Provide early warning	Prediction evaluated against failure horizon	To be experimentally verified
Create industrial decision support	End-to-end PX-204 workflow	Demonstrated as system design

4.13 Strengths & Limitations

Strengths:

- Uses contextual language modelling rather than depending only on exact keywords.
- Explicitly represents maintenance-event progression through a temporal sequence model.
- Divides the architecture into modules that can be tested independently.
- Uses a continuous PX-204 case study to connect technical model behaviour with an industrial maintenance scenario.

- Can complement structured sensor-based predictive maintenance instead of requiring it to be replaced.

Limitations:

- Performance depends on the quality and coverage of labelled maintenance narratives.
- Rare failure classes can create class imbalance and unstable estimates.
- Transformer-based models require more computation than simple rule-based systems.
- Temporal modelling must be designed carefully to prevent future-information leakage.
- The system is a decision-support tool and cannot replace physical inspection or engineering judgement.

4.14 Project Planning, Roles & Budget

Table 18: Gantt / Milestone Timeline

Activity	Planned Period	Status
Requirement analysis	Project initiation	Completed
Literature review	Planning and analysis	Completed
System architecture	Design phase	Completed
Module 1 implementation	Implementation phase	Completed
Module 2 implementation	Implementation phase	Completed
Module 3 implementation	Implementation phase	Completed
Module 4 implementation	Integration phase	Completed
Testing and validation	Testing phase	Completed
Final documentation	Reporting phase	Completed

Table 19: Project Roles

Student	Assigned Responsibility	Actual Contribution
Harissh M – 192424014	Semantic NLP	Semantic NLP
Hariharan G – 192424156	Severity classification and temporal modelling	Severity classification and temporal modelling
Nirmal Kasi – 192424248	Failure prediction, integration and evaluation	Failure prediction, integration and evaluation

Table 20: Project Budget

Item	Estimated Cost	Actual Cost
Python / open-source NLP libraries	Open-source	No separate cost
Development environment	Existing	NIL
Git / GitHub	Free	NIL
Computer / development system	Existing resource	NIL
GPU / cloud compute, if used	NIL	NIL
Internet / project expenses	NIL	NIL
TOTAL	NIL	NIL

CHAPTER 5

CONCLUSION & REFLECTION

5.1 Conclusion

The project addresses an important engineering problem in industrial maintenance: useful machine-condition information is often recorded in natural language, while many predictive systems are designed around structured measurements. The proposed framework treats technician narratives as an additional source of predictive evidence and uses NLP to convert those observations into a machine-condition representation.

Centrifugal pump PX-204 is used as the continuous case study. At the beginning of the story, a technician observes slight vibration near a bearing. A later report describes increased vibration and unusual noise, followed by a report of higher bearing temperature. Finally, severe vibration is observed. The engineering significance of these observations becomes clearer when they are connected as a sequence rather than treated as unrelated text.

The four-module architecture provides a clear division of responsibilities. RoBERTa first provides contextual understanding of maintenance language. A second RoBERTa stage classifies fault severity. BiLSTM models the chronological progression of the resulting events. The final RoBERTa prediction stage estimates failure risk from the combined evidence. This structure provides an explicit path from technician language to maintenance decision support.

The final value of the proposed system must be established experimentally. Precision, recall, F1-score, ROC-AUC, and early-warning lead time should be calculated from the actual evaluation data. The system should then be compared against suitable baselines, particularly a simpler keyword or classical model and a RoBERTa-only classifier. The resulting evidence will determine whether the added temporal stage provides a measurable improvement.

5.2 Future Work

- Fuse maintenance narratives with real-time vibration, temperature, pressure, current, and other sensor measurements.
- Extend the framework from the PX-204 case study to multiple pump types and other rotating equipment such as compressors, motors, and gearboxes.

- Develop root-cause and failure-chain analysis so that the system can suggest likely fault mechanisms in addition to predicting risk.
- Support multilingual maintenance narratives and industrial abbreviations used by different technician teams.
- Integrate the prediction layer with Computerized Maintenance Management Systems for inspection scheduling and work-order prioritization.
- Explore lightweight Transformer variants for lower-latency industrial deployment.
- Investigate LLM and Retrieval-Augmented Generation approaches for retrieving maintenance manuals, historical incidents, and troubleshooting procedures.
- Study online model monitoring and drift detection as equipment behaviour and terminology change over time.

5.3 Professional Learning & Individual Reflection

5.3.1 New Knowledge Acquired

- Transformer-based contextual language modelling using RoBERTa.
- Sequence modelling using Bidirectional LSTM networks.
- Classification metrics and evaluation of imbalanced failure data.
- Industrial predictive-maintenance concepts and failure progression.
- Model integration, testing, debugging and technical documentation.

5.3.2 Engineering & Professional Skills Developed

- Requirement analysis and modular system design.
- Breaking a real-world engineering problem into independently testable NLP components.
- Selecting algorithms based on their role rather than using models without a defined purpose.
- Responsible interpretation of model predictions in an industrial safety context.
- Team coordination, version control, technical communication and report preparation.

5.3.3 Individual Reflection — Harish M

During the project, I worked primarily on the contextual NLP stage. I learned how Transformer-based representations can capture relationships between technical terms and surrounding words. A major learning point was that maintenance text cannot be treated like ordinary conversational text because abbreviations, component names, fault descriptions and severity expressions carry specific engineering meaning. If the project were repeated, I would improve the annotation strategy and perform more systematic error analysis on technical terminology.

5.3.4 Individual Reflection — Hariharan G

During the project, I worked on fault-severity classification and temporal progression. The main challenge was connecting individual maintenance observations into a meaningful sequence. Implementing the BiLSTM stage helped demonstrate why temporal modelling is different from simply classifying each report independently. If the project were repeated, I would focus more strongly on chronological validation and methods for handling irregular time gaps between maintenance events.

5.3.5 Individual Reflection — Nirmal Kasi

During the project, I worked on the failure-risk prediction stage, integration, testing and result interpretation. This highlighted the importance of selecting evaluation metrics that reflect the engineering cost of different errors. A high overall accuracy can still hide poor failure detection when failures are rare. The project also strengthened my understanding of how an AI model should remain a decision-support component when the physical system is safety-relevant.

REFERENCES

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I., “Attention Is All You Need,” *Advances in Neural Information Processing Systems*, 2017.
- [2] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *Proceedings of NAACL-HLT*, 2019.
- [3] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V., “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” *arXiv:1907.11692*, 2019.
- [4] Hochreiter, S. and Schmidhuber, J., “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [5] Schuster, M. and Paliwal, K. K., “Bidirectional Recurrent Neural Networks,” *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.
- [6] Jardine, A. K. S., Lin, D., and Banjevic, D., “A Review on Machinery Diagnostics and Prognostics Implementing Condition-Based Maintenance,” *Mechanical Systems and Signal Processing*, vol. 20, no. 7, pp. 1483–1510, 2006.
- [7] Lei, Y., Li, N., Guo, L., Li, N., Yan, T., and Lin, J., “Machinery Health Prognostics: A Systematic Review from Data Acquisition to RUL Prediction,” *Mechanical Systems and Signal Processing*, vol. 104, pp. 799–834, 2018.
- [8] Carvalho, T. P., Soares, F. A. A. M. N., Vita, R., Francisco, R. P., Basto, J. P., and Alcalá, S. G. S., “A Systematic Literature Review of Machine Learning Methods Applied to Predictive Maintenance,” *Computers & Industrial Engineering*, vol. 137, 2019.
- [9] Susto, G. A., Schirru, A., Pampuri, S., McLoone, S., and Beghi, A., “Machine Learning for Predictive Maintenance: A Multiple Classifier Approach,” *IEEE Transactions on Industrial Informatics*, vol. 11, no. 3, pp. 812–820, 2015.
- [10] Mobley, R. K., *An Introduction to Predictive Maintenance*, 2nd ed., Butterworth-Heinemann, 2002.
- [11] [Add the actual industrial maintenance NLP paper(s) selected by the project team after completing the literature search.]
- [12] [Add the actual dataset source, licensing information, and version used for the experiments.]

APPENDICES

Appendix A Example PX-204 Maintenance Narratives

- Day 1 — “Slight vibration observed near the bearing.”
- Day 5 — “Vibration has increased and a faint unusual noise is present.”
- Day 9 — “Bearing temperature is higher than normal.”
- Day 12 — “Severe vibration occurs during operation.”

Appendix B Core Mathematical Expressions

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k}) \mathbf{V}$$

$$\mathbf{P}(\mathbf{i}|\mathbf{x}) = \exp(z_i) / \sum_j \exp(z_j)$$

$$\text{Loss} = -\sum_i y_i \log(\mathbf{P}(\mathbf{i}|\mathbf{x}))$$

$$\mathbf{f}_t = \sigma(\mathbf{W}\mathbf{f} [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}\mathbf{f})$$

$$\mathbf{i}_t = \sigma(\mathbf{W}\mathbf{i} [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}\mathbf{i})$$

$$\mathbf{c}_t = \mathbf{f}_t \times \mathbf{c}_{t-1} + \mathbf{i}_t \times \tilde{\mathbf{c}}_t$$

$$\mathbf{h}_t = \mathbf{o}_t \times \tanh(\mathbf{c}_t)$$

$$\mathbf{P}(\text{Failure}) = \text{Softmax}(\mathbf{W} \mathbf{h} + \mathbf{b})$$

Appendix C Final Experimental Results

Insert the final confusion matrix, per-class Precision, Recall, F1-score, ROC-AUC, and early-warning analysis after actual experiments.

Appendix D Software Requirements

Category	Requirement
Processor	Intel Core i3 or above; higher-end CPU recommended for local Transformer inference
RAM	8 GB minimum; 16 GB recommended for larger experiments
Storage	At least 20 GB free space for environments and model files
Operating System	Windows 10/11 or compatible Linux distribution
Programming Language	Python 3.x

Libraries	PyTorch or TensorFlow, Transformers, Scikit-learn, Pandas
Development Environment	Jupyter Notebook / VS Code
GPU	Optional but recommended for Transformer fine-tuning

Appendix E Source Code / Repository Information

1. BiLSTM with Additive Self-Attention and Multi-Task Heads

File: `src/models/bilstm_model.py`

Report Section: *Deep Learning Model Architecture / Sequence Models*

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class SelfAttention(nn.Module):
    """
    Computes additive self-attention over BiLSTM hidden states
    to capture salient qualitative failure symptoms from
    maintenance text.
    """

    def __init__(self, hidden_dim: int):
        super(SelfAttention, self).__init__()

        self.projection = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.Tanh(),
            nn.Linear(hidden_dim // 2, 1)
        )

    def forward(
        self,
        lstm_output: torch.Tensor,
        mask: torch.Tensor = None
    ):
        # lstm_output shape:
        # (batch_size, seq_len, 2 * hidden_dim)

        energy = self.projection(lstm_output).squeeze(2)
        # Shape: (batch_size, seq_len)

        if mask is not None:
            energy = energy.masked_fill(mask == 0, -1e9)

        # Attention distribution alpha_t
        weights = F.softmax(energy, dim=1)
```

```

        # Context vector:
        # c = sum(alpha_t * h_t)
        context = torch.bmm(
            weights.unsqueeze(1),
            lstm_output
        ).squeeze(1)

        return context, weights

class BiLSTMAttentionClassifier(nn.Module):
    """
    2-Layer Bidirectional LSTM with Additive Attention
    and Multi-Task Output Heads.
    """

    def __init__(
        self,
        vocab_size: int,
        embed_dim: int = 128,
        hidden_dim: int = 128,
        num_layers: int = 2,
        num_failure_classes: int = 6,
        num_risk_classes: int = 4,
        dropout: float = 0.3,
        pad_idx: int = 0
    ):
        super(BiLSTMAttentionClassifier, self).__init__()

        self.embedding = nn.Embedding(
            vocab_size,
            embed_dim,
            padding_idx=pad_idx
        )

        self.lstm = nn.LSTM(
            embed_dim,
            hidden_dim,
            num_layers=num_layers,
            bidirectional=True,
            batch_first=True,
            dropout=dropout if num_layers > 1 else 0
        )

        self.lstm_out_dim = hidden_dim * 2

        self.attention = SelfAttention(
            self.lstm_out_dim
        )

        self.shared_fc = nn.Sequential(
            nn.Linear(self.lstm_out_dim, 128),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Dropout(dropout)
        )

```

```

# Multi-Task Prediction Heads
self.failure_head = nn.Linear(
    128,
    num_failure_classes
)

self.risk_head = nn.Linear(
    128,
    num_risk_classes
)

self.early_warn_head = nn.Linear(
    128,
    1
)

def forward(
    self,
    input_ids: torch.Tensor,
    mask: torch.Tensor = None
):
    embedded = self.embedding(input_ids)

    lstm_out, _ = self.lstm(embedded)

    context, attn_weights = self.attention(
        lstm_out,
        mask
    )

    shared_rep = self.shared_fc(context)

    return {
        "failure_logits":
            self.failure_head(shared_rep),

        "risk_logits":
            self.risk_head(shared_rep),

        "early_warn_logits":
            self.early_warn_head(
                shared_rep
            ).squeeze(-1),

        "attention_weights":
            attn_weights,

        "embedding_repr":
            context
    }

```

2. BERT and RoBERTa Multi-Task Transformer Classifier

Files: src/models/bert_model.py / src/models/roberta_model.py

Report Section: *Transformer Architectures / Transfer Learning*

```
import torch
import torch.nn as nn
from transformers import AutoModel, AutoConfig

class MaintenanceTransformerClassifier(nn.Module):
    """
    Fine-tuned Transformer Classifier using BERT or RoBERTa
    with Multi-Task Diagnostic Heads.
    """

    def __init__(
        self,
        pretrained_model_name: str = "bert-base-uncased",
        num_failure_classes: int = 6,
        num_risk_classes: int = 4,
        dropout: float = 0.2
    ):
        super(MaintenanceTransformerClassifier, self).__init__()

        self.config = AutoConfig.from_pretrained(
            pretrained_model_name
        )

        self.transformer = AutoModel.from_pretrained(
            pretrained_model_name
        )

        hidden_size = self.config.hidden_size
        # Typically 768 dimensions for base models

        self.dropout = nn.Dropout(dropout)

        self.dense = nn.Linear(
            hidden_size,
            256
        )

        self.activation = nn.GELU()

        # Parallel Task Heads
        self.failure_head = nn.Linear(
            256,
            num_failure_classes
        )

        self.risk_head = nn.Linear(
            256,
            num_risk_classes
        )

        self.early_warn_head = nn.Linear(
            256,
```



```

        1
    )

def forward(
    self,
    input_ids: torch.Tensor,
    attention_mask: torch.Tensor = None
):

    outputs = self.transformer(
        input_ids=input_ids,
        attention_mask=attention_mask
    )

    # Extract sentence-level contextual representation
    if (
        hasattr(outputs, "pooler_output")
        and outputs.pooler_output is not None
    ):
        cls_repr = outputs.pooler_output
    else:
        cls_repr = outputs.last_hidden_state[:, 0, :]

    x = self.dropout(cls_repr)

    x = self.activation(
        self.dense(x)
    )

    return {
        "failure_logits":
            self.failure_head(x),

        "risk_logits":
            self.risk_head(x),

        "early_warn_logits":
            self.early_warn_head(x).squeeze(-1),

        "embedding_repr":
            cls_repr
    }

```

3. Temporal Degradation LSTM for Multi-Visit Trajectories

File: src/models/temporal_lstm.py

Report Section: *Prognostics and Remaining Useful Life (RUL) Modeling*

```

import torch
import torch.nn as nn

```

```

class TemporalDegradationLSTM(nn.Module):
    """

```

Models multi-visit equipment degradation histories
 [v1, v2, ..., vT] over time by fusing NLP narrative
 embeddings with continuous sensor telemetry.
 """

```
def __init__(
    self,
    text_embed_dim: int = 128,
    telemetry_dim: int = 4,
    hidden_dim: int = 64,
    num_layers: int = 2,
    num_failure_classes: int = 6,
    num_risk_classes: int = 4,
    dropout: float = 0.25
):
    super(TemporalDegradationLSTM, self).__init__()

    # 128 text dimensions + 4 telemetry dimensions
    fusion_dim = text_embed_dim + telemetry_dim

    self.lstm = nn.LSTM(
        input_size=fusion_dim,
        hidden_size=hidden_dim,
        num_layers=num_layers,
        batch_first=True,
        bidirectional=False,
        dropout=dropout if num_layers > 1 else 0
    )

    # Temporal Attention over inspection visits
    self.time_attention = nn.Sequential(
        nn.Linear(hidden_dim, 32),
        nn.Tanh(),
        nn.Linear(32, 1)
    )

    self.fc = nn.Sequential(
        nn.Linear(hidden_dim, 64),
        nn.ReLU(),
        nn.Dropout(dropout)
    )

    # Classification and RUL Regression Heads
    self.failure_head = nn.Linear(
        64,
        num_failure_classes
    )

    self.risk_head = nn.Linear(
        64,
        num_risk_classes
    )

    # Time-to-Failure prediction in days
    self.ttf_head = nn.Linear(
        64,
        1
```

```

    )

def forward(
    self,
    visit_sequences: torch.Tensor,
    seq_lengths: torch.Tensor = None
):
    # visit_sequences shape:
    # (batch_size, num_visits, 132)

    lstm_out, _ = self.lstm(
        visit_sequences
    )

    # Attention scores across time visits
    attn_scores = self.time_attention(
        lstm_out
    ).squeeze(-1)

    # Attention weights
    attn_weights = torch.softmax(
        attn_scores,
        dim=1
    )

    # Temporal context vector
    context = torch.bmm(
        attn_weights.unsqueeze(1),
        lstm_out
    ).squeeze(1)

    features = self.fc(context)

    return {
        "failure_logits":
            self.failure_head(features),

        "risk_logits":
            self.risk_head(features),

        "ttf_pred":
            self.ttf_head(features).squeeze(-1),

        "temporal_attention_weights":
            attn_weights
    }

```

4. Industrial NLP Preprocessing and Domain Acronym Expansion

File: src/preprocessing.py

Report Section: *Data Preprocessing and Domain-Specific Normalization*

```
import re
```

```

# Industrial equipment domain abbreviation mappings
DOMAIN_ABBREVIATIONS = {

    r"\bvib\b|\bvibs\b":
        "vibration",

    r"\bbrg\b|\bbrgs\b":
        "bearing",

    r"\bde\b":
        "drive end",

    r"\bnde\b":
        "non drive end",

    r"\bpm\b":
        "preventative maintenance",

    r"\bcm\b":
        "corrective maintenance",

    r"\brtd\b|\brtds\b":
        "resistance temperature detector",

    r"\brpm\b":
        "revolutions per minute",

    r"\bpsi\b":
        "pounds per square inch",

    r"\bgmf\b":
        "gear mesh frequency",

    r"\bmtr\b":
        "motor",

    r"\btemp\b|\btemps\b":
        "temperature",

    r"\bpress\b":
        "pressure",

    r"\bo-ring\b|\boring\b":
        "o ring seal"
}

def clean_maintenance_text(text: str) -> str:
    """
    Standardizes noisy industrial maintenance text by
    expanding shorthand, normalizing numerical measurements,
    and removing extraneous symbols.
    """

    if not isinstance(text, str):

```

```

        return ""

text = text.lower().strip()

# Expand technical abbreviations
for pattern, expansion in DOMAIN_ABBREVIATIONS.items():
    text = re.sub(
        pattern,
        expansion,
        text
    )

# Standardize common telemetry units
# Example: 4.5mm/s -> 4.5 mm/s
text = re.sub(
    r"(\d+(?:\.\d+)?)\s*(mm/s|ips|hz|c|f|psi|rpm|bar|g)",
    r"\1 \2",
    text
)

# Remove unwanted punctuation while retaining
# decimal numbers and hyphens
text = re.sub(
    r"^[^w\s\.\-\/]",
    "",
    text
)

# Remove extra whitespace
text = re.sub(
    r"\s+",
    " ",
    text
).strip()

return text

```

5. Multi-Task Loss Formulation and Shannon Entropy Uncertainty

Files: `src/train.py / src/models/hybrid_pipeline.py`

Report Section: *Loss Optimization and Model Uncertainty Formulation*

```

import torch
import torch.nn as nn
import numpy as np

class MultiTaskMaintenanceLoss(nn.Module):
    """
    Joint Multi-Task Loss Function optimizing:

    1. Failure Mode Classification
    2. Risk Tier Estimation
    3. Early Warning Detection
    """

```

```

"""

def __init__(
    self,
    alpha: float = 1.0,
    beta: float = 0.8,
    gamma: float = 0.5
):
    super(MultiTaskMaintenanceLoss, self).__init__()

    # Loss weights
    self.alpha = alpha
    self.beta = beta
    self.gamma = gamma

    self.ce_loss = nn.CrossEntropyLoss()

    self.bce_loss = nn.BCEWithLogitsLoss()

def forward(
    self,
    preds: dict,
    targets: dict
):
    # Failure Mode Loss
    loss_failure = self.ce_loss(
        preds["failure_logits"],
        targets["failure_mode"]
    )

    # Risk Tier Loss
    loss_risk = self.ce_loss(
        preds["risk_logits"],
        targets["risk_level"]
    )

    # Early Warning Loss
    loss_early_warn = self.bce_loss(
        preds["early_warn_logits"],
        targets["early_warning"].float()
    )

    # Total Multi-Task Loss
    total_loss = (
        (self.alpha * loss_failure)
        + (self.beta * loss_risk)
        + (self.gamma * loss_early_warn)
    )

    return total_loss

def calculate_shannon_entropy(
    probabilities: np.ndarray
) -> float:
    """

```

Computes Shannon Entropy to quantify prediction uncertainty across classes.

```
H(p) = -sum(p_i * log2(p_i))  
"""
```

```
eps = 1e-12
```

```
probs = np.clip(  
    probabilities,  
    eps,  
    1.0 - eps  
)
```

```
entropy = -np.sum(  
    probs * np.log2(probs)  
)
```

```
return round(  
    float(entropy),  
    4  
)
```

Repository: <https://github.com/Nirmal-kk/NLP-Capstone.git>